

AI-Driven Scheme Matching for Marginalized Entrepreneurs

Solution Architecture & Implementation Document

1. Problem Statement Recap

NSFDC (National Scheduled Castes Finance and Development Corporation) provides concessional financial assistance to SC-community beneficiaries with annual family income up to ₹5 lakh, covering up to 90% of project/education cost at 6.5%–8% interest. Funds are not disbursed directly — they route through a Channel Finance System of 100+ partners (SCAs, PSBs, RRBs, NBFC-MFIs).

Core failure modes:

- Citizens don't know which scheme fits them (Micro Credit $\leq$ ₹1.4L / Term Loan $\leq$ ₹50L / Educational Loan).
- Citizens can't identify the right, capable Channel Partner nearby.
- Applications get misrouted to partners that are backlogged, capital-constrained, or have high NPAs/overdues — causing delay and rejection.
- No visibility for NSFDC into how well the network is actually performing.
- Beyond matching: citizens don't know if they're actually *ready* to apply, documents are often invalid without anyone noticing, projects proposed are sometimes financially unrealistic, and applicants get little visibility or support once they've applied.

2. Solution Philosophy

This is framed as an **allocation, trust, and readiness problem** — not a lookup problem. The platform is a **scheme-to-disbursement copilot** that:

- predicts real approval chances and readiness before submission (not just eligibility),
- routes citizens to the partner most likely to serve them well,
- removes document and identity friction at the source,
- grounds every recommendation in cited, versioned policy — never a hallucinated answer,
- works for people without reliable internet, smartphones, or digital literacy,
- stays with the applicant after disbursement, not just up to it,
- gives NSFDC a live, auditable view of the whole network.

3. Architecture Overview — Six Layers

Layer	Purpose
A. Trust & Risk	Matching, routing, and predicting outcomes before submission
B. Integration & Data Quality	Removing document/identity/partner-status friction
C. Governance & Evidence	Making every AI output sourced, versioned, and auditable
D. Inclusion & Accessibility	Reaching users with low connectivity, literacy, or digital access
E. Post-Application Support	Keeping applicants informed and protected after they apply and after disbursement
F. Observability	Giving NSFDC a live, auditable view of the whole system

4. Full Feature Set by Layer

Layer A — Trust & Risk

1. **Smart Scheme Recommender** — explainable rule-engine scoring against scheme criteria
2. **Dynamic EMI / Financial Calculator** — interest bands, moratorium, subsidy comparison
3. **Geo-Spatial Partner Locator & Router**
4. **Capacity-Aware Routing Engine** — optimization-based, avoids overloaded/high-NPA partners
5. **Application-Success Probability Model** — predicts approval likelihood + top blocking factor
6. **Scheme-Bundling / Cost-of-Capital Optimizer** — stackable schemes, blended interest rate
7. **Livelihood Project Viability Simulator** (*new*) — revenue/expense/break-even/repayment-risk modeling so eligible-but-unviable projects are caught before submission

8. **Application-Readiness Engine** (*new*) — the orchestrator: combines scheme eligibility + document readiness + project viability + partner capacity into a single "ready to apply / not yet" verdict with an exact action checklist

Layer B — Integration & Data Quality

9. **Document Readiness and Consent Vault** (*expanded*) — detects missing, blurry, expired, duplicate, or mismatched documents; manages granular, consent-based document sharing with partners
10. **DigiLocker / Aadhaar e-KYC Integration**
11. **Duplicate / Multi-Partner Application Flagging**
12. **Serviceability and Capacity Feedback Loop** (*new*) — tracks whether a partner office is actually functional, verified, and accepting applications right now, combining admin verification with user feedback

Layer C — Governance & Evidence

13. **Policy-as-Code with Versioned Rules** (*new*) — every scheme rule, loan limit, interest rate, and document requirement stored as a dated, source-linked, version-controlled policy object with approval history and rollback
14. **Evidence-First Scheme Reasoning** (*new*) — every recommendation is returned with its source document, exact rule, version, page reference, effective date, and a confidence score — structurally prevents hallucinated loan limits or rates rather than fact-checking after generation

Layer D — Inclusion & Accessibility

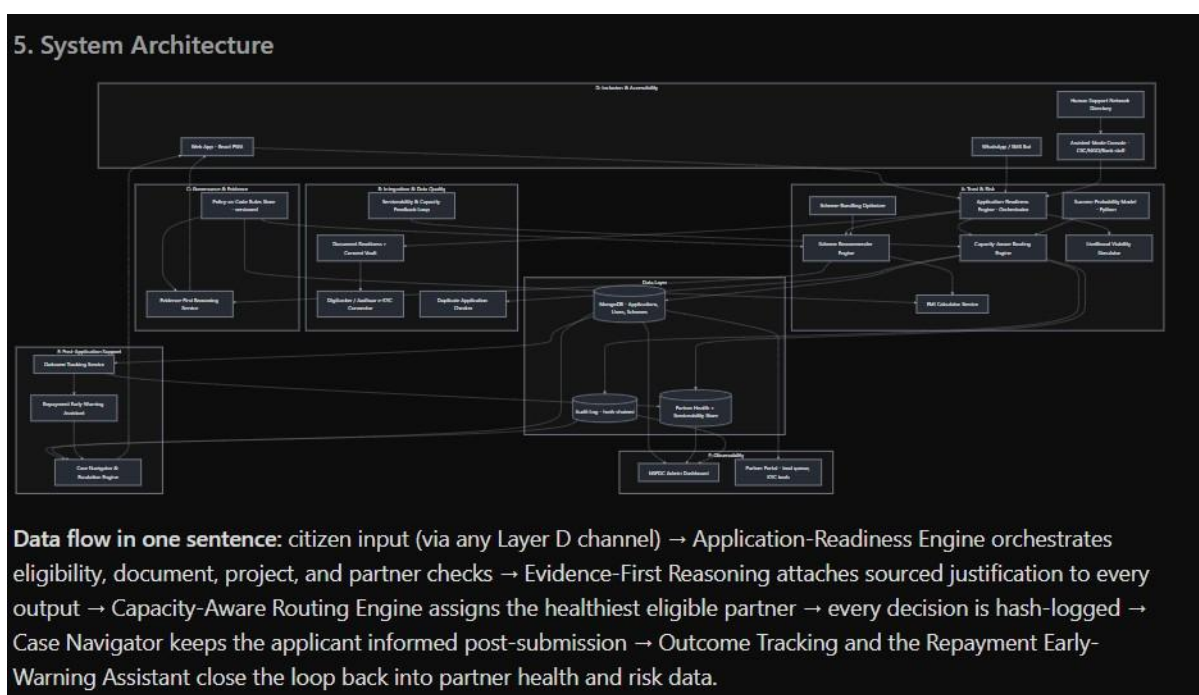
15. **Offline-First and Assisted Mode** (*new*) — forms, EMI calculation, and data entry work offline and sync later; CSC operators, NGOs, and bank staff can operate the platform on a citizen's behalf
16. **WhatsApp / SMS Fallback Channel**
17. **Human-in-the-Loop Social Support Network** (*new*) — connects applicants to trained CSC operators, NGOs, welfare officers, and community facilitators rather than assuming full digital independence
18. **Vernacular / voice interface** (Bhashini-based)

Layer E — Post-Application Support

19. **Case Navigator and Escalation Engine** (*new*) — timelines, reminders, pending-document alerts, delay notifications, grievance registration, escalation support
20. **Closed-Loop Post-Disbursement Outcome Tracking**
21. **Repayment Early-Warning Assistant** (*new*) — consent-based monitoring of repayment/financial indicators to catch stress before default, with reminders, restructuring info, and human support

Layer F — Observability

22. **NSFDC Analytics Dashboard** — utilization heatmap, demand forecast, partner ranking
23. **Immutable Audit Trail** — hash-chained log on every routing/status decision, feeding the Case Navigator and the dashboard



6. Component Detail

6.1 Application-Readiness Engine (new — orchestration layer)

- **Logic:** calls the Recommender, Document Vault, Viability Simulator, and Partner Serviceability Store in parallel, then returns a single readiness verdict ("Ready to apply" / "3 items remaining") with an exact checklist of blocking items.
- **Why it matters:** existing platforms (including myScheme) tell you what you qualify for, not whether you can actually submit successfully today. This is the feature that converts "informational" into "actionable."
- **Stack:** Node.js orchestrator service using Promise.all fan-out to the four dependent services; result cached per-applicant-session.

6.2 Evidence-First Scheme Reasoning (new)

- **Logic:** every recommendation returned by the Recommender or Bundling Optimizer is enriched with {source_document, rule_id, version, page_reference, effective_date, confidence_score} pulled directly from the Policy-as-Code store — not generated freeform text.
- **Why this avoids hallucination by design:** because it never generates numbers from a language model — it looks them up from a structured, versioned rule store and simply explains the lookup. This is a meaningful architectural distinction from a RAG chatbot over unstructured PDFs, and worth explaining clearly to judges.
- **Stack:** Node.js service reading directly from the Policy-as-Code store; no generative model in the numeric path at all.

6.3 Policy-as-Code with Versioned Rules (new)

- **Logic:** scheme rules, loan limits, interest rates, and document requirements are stored as dated, source-linked JSON policy objects with an approval history and rollback capability — a git-like version chain rather than a mutable config file.
- **Stack:** MongoDB collection modeled as an append-only version chain (policy_id, version, effective_date, source_ref, approved_by, superseded_by), or literally backed by a Git repository of YAML policy files for a demonstrably auditable history.

6.4 Livelihood Project Viability Simulator (new)

- **Logic:** takes proposed project type, expected revenue, recurring expenses, and seasonality, and computes monthly surplus, break-even period, EMI affordability ratio, and a repayment-risk flag under optimistic/base/conservative scenarios.
- **Stack:** Node.js or Python numeric module; output feeds both the Application-Readiness Engine and the Success-Probability Model as an additional risk feature.

6.5 Serviceability and Capacity Feedback Loop (new)

- **Logic:** maintains a partner-status registry separate from raw NPA/fund data — is_accepting_applications, last_verified_date, verified_by, user_feedback_rating, admin_flag. Community-reported and admin-verified, feeding the Routing Engine as a real-time gating signal alongside financial health.
- **Stack:** MongoDB collection + a lightweight moderation endpoint for admin verification; simple star-rating submission from applicants post-visit.

6.6 Document Readiness and Consent Vault (expanded)

- **Logic:** OCR field extraction plus **image-quality checks (blur detection via Laplacian variance)**, **expiry-date parsing**, and **perceptual-hash duplicate detection**, layered on top of the original field-presence validation. Adds a **consent**

ledger: applicants grant granular, revocable permission for which partner can see which document, logged into the audit trail.

- **Stack:** Tesseract OCR + OpenCV for blur/quality checks, imagehash-style perceptual hashing for duplicate detection, consent events written to the same hash-chained audit log as routing decisions.

6.7 Offline-First and Assisted Mode (new)

- **Logic:** the citizen-facing app is a Progressive Web App — forms, EMI calculation, and document metadata entry work fully offline using local storage, syncing automatically when connectivity returns. A separate **Assisted-Mode Console** lets a CSC operator, NGO worker, or bank staff member log in and complete the flow on an applicant's behalf, with the applicant's consent logged.
- **Stack:** Service Workers + IndexedDB (via Workbox) for offline capability; a role-based login (assisted_by field on every record) for the Assisted-Mode Console, reusing the same Core Application Layer.

6.8 Human-in-the-Loop Social Support Network (new)

- **Logic:** a directory of trained CSC operators, NGOs, welfare officers, and community facilitators that applicants (or the Assisted-Mode Console) can be routed to, especially for elderly, disabled, illiterate, or digitally inexperienced users.
- **Stack:** MongoDB directory collection with location + specialization tags; surfaced in the app wherever a readiness blocker requires human help rather than a digital fix.

6.9 Case Navigator and Escalation Engine (new)

- **Logic:** gives every applicant a visible timeline of their application, scheduled reminders for pending documents, delay notifications once an SLA is breached (using the same audit trail data), and a grievance registration flow that auto-escalates unresolved cases.
- **Stack:** Node.js service + a scheduler (node-cron or a Bull/Redis queue) for reminders and SLA-breach checks; grievance tickets stored in MongoDB and surfaced on both the applicant's timeline and the NSFDC dashboard.

6.10 Repayment Early-Warning Assistant (new)

- **Logic:** post-disbursement, and only with explicit applicant consent, tracks repayment behavior and self-reported financial indicators to flag early signs of stress (missed reminder responses, self-reported income drop), triggering preventive reminders, restructuring information, and a handoff to the Human Support Network before default occurs.
- **Stack:** scheduled monitoring job reading from the Outcome Tracking Service; rule-based risk flags (not a black-box model, for the same explainability reasons as

the Recommender); all monitoring explicitly gated behind a consent flag stored in the Consent Vault.

(Components carried over unchanged from the original design — Smart Scheme Recommender, EMI Calculator, Geo-Spatial Locator, Capacity-Aware Routing Engine, Success-Probability Model, Scheme-Bundling Optimizer, DigiLocker/e-KYC, Duplicate Flagging, WhatsApp/SMS channel, NSFDC Dashboard, Audit Trail, Outcome Tracking — retain the detail already established for them; see Section 4 for their one-line role in the layered structure.)

7. Full Tech Stack Summary

Layer	Technology
Frontend (citizen, offline-capable)	React PWA, Tailwind, Workbox, IndexedDB
Frontend (assisted-mode, admin, partner)	React, Recharts/Chart.js
Core backend	Node.js, Express
ML microservice	Python, scikit-learn, FastAPI
Database	MongoDB
Maps	Mapbox / Google Maps API (OpenStreetMap fallback)
OCR + document quality	Tesseract OCR, OpenCV (blur detection), perceptual hashing (duplicate detection)
Identity	DigiLocker Partner API, UIDAI e-KYC (sandboxed/mock)
Messaging	Twilio WhatsApp Business API
Vernacular/voice	Bhashini APIs
Policy/rules store	MongoDB version-chain or Git-backed YAML policies
Scheduling/reminders	node-cron / Bull + Redis
Audit log	Hash-chained append-only MongoDB collection

Layer	Technology
Hosting (demo)	Vercel/Render (frontend + backend), MongoDB Atlas

8. Implementation Plan (Build Order)

Phase 1 — Trust & Risk core

1. Policy-as-Code Rules Store (build this early — everything else reads from it)
2. Scheme Recommender + EMI Calculator (reading from Policy-as-Code)
3. Evidence-First Reasoning wrapper on top of the Recommender
4. Partner Health Store + Capacity-Aware Routing Engine
5. Application-Readiness Engine (orchestrator — needs 2–4 done first)

Phase 2 — Data quality & integration 6. Document Readiness and Consent Vault (start with field + blur checks, add duplicate hashing if time allows) 7. Serviceability and Capacity Feedback Loop 8. Duplicate Application Flagging 9. DigiLocker/e-KYC connector (mocked)

Phase 3 — Prediction & viability 10. Success-Probability Model (Python microservice) 11. Livelihood Project Viability Simulator 12. Scheme-Bundling Optimizer

Phase 4 — Post-application support 13. Audit Trail (needed before Case Navigator) 14. Case Navigator and Escalation Engine 15. Outcome Tracking + Repayment Early-Warning Assistant

Phase 5 — Observability & reach 16. NSFDC Analytics Dashboard 17. WhatsApp/SMS bot, Offline-First PWA behavior, Assisted-Mode Console, Human Support Network directory, vernacular/voice layer

This order still front-loads the differentiator (Policy-as-Code → Recommender → Evidence-First Reasoning → Routing Engine → Readiness Engine) so the most defensible, hardest-to-copy part of the system is demoable first, even under time pressure.

9. What's Real vs. Simulated in the Demo

Component	Demo status
Recommender, EMI calculator, routing engine, audit trail, readiness engine	Fully functional, real logic

Component	Demo status
Policy-as-Code store	Real, seeded with actual published NSFDC scheme rules
Partner fund/NPA/serviceability data	Synthetic but realistic dataset, clearly labeled
Success-probability model, viability simulator	Real mechanism, trained/tested on synthetic historical data
DigiLocker / Aadhaar e-KYC	Sandbox/mockd — production access requires government empanelment
WhatsApp bot, offline sync	Functional on Twilio sandbox / local PWA testing
Repayment Early-Warning Assistant	Rule logic real; live repayment data simulated

Being explicit about this in the presentation builds credibility — judges expect simulated data in a hackathon build and penalize teams who imply things are live when they aren't.

10. Impact Metrics to Present

- **Reduction in average disbursement time** — routing engine avoiding backlog-prone partners
- **Reduction in misrouted/rejected applications** — pre-validation + readiness engine catching issues before submission
- **Improvement in fund utilization rate** — capacity-aware routing preventing over-concentration
- **Reduction in application abandonment** — Case Navigator visibility and reminders
- **Reduction in defaults / repayment distress** — Early-Warning Assistant catching stress before missed payments
- **Reach extension** — WhatsApp/SMS, offline mode, and Assisted Mode extending access to non-smartphone, low-connectivity, and low-literacy users

11. Future Scope

- Full production integration with DigiLocker/UIDAI once empanelled
- Real-time partner API integration for live fund corpus updates
- Expansion to other channel-finance schemes beyond NSFDC (SC/ST/OBC/Minority corporations follow similar models)
- Bank-statement-based (consent-driven) repayment risk scoring, replacing self-reported indicators